

11. Implement a C program to perform symbol table operations.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int cnt = 0;

struct symtab
{
    char label[20];
    int addr;
} sy[50];

void insert();
int search(char *);
void display();
void modify();

int main()
{
    int ch, val;
    char lab[10];

    do
    {
        printf("\n1.Insert\n2.Display\n3.Search\n4.Modify\n5.Exit\n");
        scanf("%d", &ch);

        switch (ch)
        {
            case 1:
                insert();
                break;

            case 2:
                display();
                break;

            case 3:
                printf("Enter the label: ");
                scanf("%s", lab);

                val = search(lab);

                if (val == 1)
                    printf("Label is found");
                else
                    printf("Label is not found");
```

```

        break;

    case 4:
        modify();
        break;

    case 5:
        exit(0);
        break;
    }

} while (ch < 5);

return 0;
}

void insert()
{
    int val;
    char lab[10];

    printf("Enter the label: ");
    scanf("%s", lab);

    val = search(lab);

    if (val == 1)
    {
        printf("Duplicate symbol");
    }
    else
    {
        strcpy(sy[cnt].label, lab);

        printf("Enter the address: ");
        scanf("%d", &sy[cnt].addr);

        cnt++;
    }
}

int search(char *s)
{
    int flag = 0, i;

    for (i = 0; i < cnt; i++)
    {
        if (strcmp(sy[i].label, s) == 0)
        {
            flag = 1;
        }
    }
}

```

```

    return flag;
}

void modify()
{
    int val, ad, i;
    char lab[10];

    printf("Enter the label: ");
    scanf("%s", lab);

    val = search(lab);

    if (val == 0)
    {
        printf("No such symbol");
    }
    else
    {
        printf("Label is found\n");

        printf("Enter the address: ");
        scanf("%d", &ad);

        for (i = 0; i < cnt; i++)
        {
            if (strcmp(sy[i].label, lab) == 0)
            {
                sy[i].addr = ad;
            }
        }
    }
}

void display()
{
    int i;

    for (i = 0; i < cnt; i++)
    {
        printf("%s\t%d\n", sy[i].label, sy[i].addr);
    }
}

```

Output:

```
1.Insert
2.Display
3.Search
4.Modify
5.Exit
1
Enter the label: a
Enter the address: 100

1.Insert
2.Display
3.Search
4.Modify
5.Exit
2
a 100

1.Insert
2.Display
3.Search
4.Modify
5.Exit
3
Enter the label: a
Label is found

1.Insert
2.Display
3.Search
4.Modify
5.Exit
4
Enter the label: a
Label is found
Enter the address: 200

1.Insert
2.Display
3.Search
4.Modify
5.Exit
2
a 200

1.Insert
2.Display
3.Search
4.Modify
5.Exit
5

Process finished with exit code 0
```

12. Write a C program to construct recursive descent parsing for the given grammar

$E \rightarrow TE'$
 $E' \rightarrow +TE' / \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' / \epsilon$
 $F \rightarrow (E) / id$

Program:

```
#include <stdio.h>
#include <string.h>

char input[100];
int i = 0;
```

```

int E();
int EP();
int T();
int TP();
int F();

int main()
{
    printf("\nRecursive Descent Parsing for the following grammar\n");
    printf("\nE -> TE'\n");
    printf("E' -> +TE' | @'\n");
    printf("T -> FT'\n");
    printf("T' -> *FT' | @'\n");
    printf("F -> (E) | ID'\n");

    printf("\nEnter the string to be checked: ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    if (E())
    {
        if (input[i] == '\0')
            printf("\nString is accepted.\n");
        else
            printf("\nString is not accepted.\n");
    }
    else
    {
        printf("\nString is not accepted.\n");
    }

    return 0;
}

int E()
{
    if (T())
    {
        if (EP())
            return 1;
    }
    return 0;
}

int EP()
{
    if (input[i] == '+')
    {
        i++;
    }
}

```

```

    if (T())
    {
        if (EP())
            return 1;
    }

    return 0;
}

return 1;
}

```

```

int T()
{
    if (F())
    {
        if (TP())
            return 1;
    }

    return 0;
}

```

```

int TP()
{
    if (input[i] == '*')
    {
        i++;

        if (F())
        {
            if (TP())
                return 1;
        }

        return 0;
    }

    return 1;
}

```

```

int F()
{
    if (input[i] == '(')
    {
        i++;

        if (E())
        {
            if (input[i] == ')')
            {
                i++;

```

```

        return 1;
    }
}

return 0;
}
else if ((input[i] >= 'a' && input[i] <= 'z') ||
        (input[i] >= 'A' && input[i] <= 'Z'))
{
    i++;
    return 1;
}

return 0;
}

```

Output:

Recursive Descent Parsing for the following grammar

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid @$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid @$

$F \rightarrow (E) \mid ID$

Enter the string to be checked: $(a+b)*c$

String is accepted.

Process finished with exit code 0

13. Write a C program to implement either Top Down parsing technique or Bottom Up Parsing technique to check whether the given input string is satisfying the grammar or not.

Program:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char string[50];
    int flag, count = 0;

    printf("The grammar is: S -> aS | Sb | ab\n");
    printf("Enter the string to be checked:\n");

    fgets(string, sizeof(string), stdin);
    string[strcspn(string, "\n")] = '\0';

    if (string[0] == 'a')
    {
        flag = 0;

        for (count = 1; string[count] != '\0'; count++)
        {
            if (string[count] == 'b')
            {
                flag = 1;
            }
            else if ((flag == 1) && (string[count] == 'a'))
            {
                printf("The string does not belong to the specified grammar.\n");
                return 0;
            }
            else if (string[count] == 'a')
            {
                continue;
            }
            else
            {
                printf("The string does not belong to the specified grammar.\n");
                return 0;
            }
        }

        if (flag == 1)
            printf("String accepted.\n");
        else
            printf("String not accepted.\n");
    }
}
```



```

else
{
    printf("String not accepted.\n");
}

return 0;
}

```

Output:

```

The grammar is: S -> aS | Sb | ab
Enter the string to be checked:
abb
String accepted.

Process finished with exit code 0

```

14. Implement the concept of Shift reduce parsing in C Programming.

Program:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char ip_sym[15], stack[15];
int ip_ptr = 0, st_ptr = 0, len, i;
char temp[2], temp2[2];
char act[15];

void check();

int main()
{
    printf("\n\t\tSHIFT REDUCE PARSER\n");
    printf("\nGrammar:");
    printf("\nE -> E+E");
    printf("\nE -> E/E");
    printf("\nE -> E*E");
    printf("\nE -> a");
    printf("\nE -> b");
}

```

```
printf("\n\nEnter the input symbol: ");
fgets(ip_sym, sizeof(ip_sym), stdin);
ip_sym[strcspn(ip_sym, "\n")] = '\0';
```

```
printf("\nStack Implementation Table");
printf("\n-----");
printf("\nStack\t\tInput\t\tAction");
printf("\n-----");
```

```
printf("\n$\t\t%s$\t\t--", ip_sym);
```

```
strcpy(act, "Shift ");
temp[0] = ip_sym[ip_ptr];
temp[1] = '\0';
strcat(act, temp);
```

```
len = strlen(ip_sym);
```

```
for (i = 0; i < len; i++)
{
    stack[st_ptr] = ip_sym[ip_ptr];
    stack[st_ptr + 1] = '\0';
```

```
    ip_sym[ip_ptr] = ' ';
    ip_ptr++;
```

```
    printf("\n%s$\t\t%s$\t\t%s", stack, ip_sym, act);
```

```
    strcpy(act, "Shift ");
    temp[0] = ip_sym[ip_ptr];
    temp[1] = '\0';
    strcat(act, temp);
```

```
    check();
    st_ptr++;
}
```

```
st_ptr++;
check();
```

```
return 0;
```

```
}
```

```
void check()
```

```
{
    int flag = 0;
```

```
    temp2[0] = stack[st_ptr];
    temp2[1] = '\0';
```

```
/* Reduce a or b to E */
```

```

if ((strcmp(temp2, "a") == 0) || (strcmp(temp2, "b") == 0))
{
    stack[st_ptr] = 'E';

    if (strcmp(temp2, "a") == 0)
        printf("\n%s\t\t%s$\t\tE -> a", stack, ip_sym);
    else
        printf("\n%s\t\t%s$\t\tE -> b", stack, ip_sym);

    flag = 1;
}

/* Reduce E+E, E*E, E/E to E */
if ((strcmp(stack, "E+E") == 0) ||
    (strcmp(stack, "E*E") == 0) ||
    (strcmp(stack, "E/E") == 0))
{
    if (strcmp(stack, "E+E") == 0)
        printf("\n$E\t\t%s$\t\tE -> E+E", ip_sym);
    else if (strcmp(stack, "E*E") == 0)
        printf("\n$E\t\t%s$\t\tE -> E*E", ip_sym);
    else
        printf("\n$E\t\t%s$\t\tE -> E/E", ip_sym);

    strcpy(stack, "E");
    st_ptr = 0;
    flag = 1;
}

if ((strcmp(stack, "E") == 0) && (ip_ptr == len))
{
    printf("\n%s\t\t%s$\t\tACCEPT\n", stack, ip_sym);
    exit(0);
}

if (flag == 0 && ip_ptr == len)
{
    printf("\n%s\t\t%s$\t\tREJECT\n", stack, ip_sym);
    exit(0);
}
}

```

Output:

```
SHIFT REDUCE PARSER

Grammar:
E -> E+E
E -> E/E
E -> E*E
E -> a
E -> b

Enter the input symbol: a+b

Stack Implementation Table
-----
Stack      Input      Action
-----
$          a+b$      --
$a         +b$       Shift a
$E         +b$       E -> a
$E+        b$        Shift +
$E+b       $          Shift b
$E+E       $          E -> b
$E         $          E -> E+E
$E         $          ACCEPT
|
Process finished with exit code 0
```

15. Write a C Program to implement the operator precedence parsing.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char *input;
int i = 0;
char lasthandle[6], stack[50], handles[][5] = {"E(", "E*E", "E+E", "i", "E^E"};
// (E) becomes )E( when pushed to stack

int top = 0, l;

char prec[9][9] =
{
    /* Input Symbols
       + - * / ^ i ( ) $ */

    /* + */ { '>', '>', '<', '<', '<', '<', '<', '>', '>' },
    /* - */ { '>', '>', '<', '<', '<', '<', '<', '>', '>' },
    /* * */ { '>', '>', '>', '>', '<', '<', '<', '>', '>' },
    /* / */ { '>', '>', '>', '>', '<', '<', '<', '>', '>' },
    /* ^ */ { '>', '>', '>', '>', '<', '<', '<', '>', '>' },
    /* i */ { '>', '>', '>', '>', '>', 'e', 'e', '>', '>' },
    /* ( */ { '<', '<', '<', '<', '<', '<', '<', '>', 'e' },
    /* ) */ { '>', '>', '>', '>', '>', 'e', 'e', '>', '>' },
    /* $ */ { '<', '<', '<', '<', '<', '<', '<', '<', '>' }
};

int getindex(char c)
{
    switch (c)
    {
        case '+': return 0;
        case '-': return 1;
        case '*': return 2;
        case '/': return 3;
        case '^': return 4;
        case 'i': return 5;
        case '(': return 6;
        case ')': return 7;
        case '$': return 8;
    }
    return -1;
}

int shift()
```

```

{
    stack[++top] = input[i++];
    stack[top + 1] = '\0';
    return 1;
}

int reduce()
{
    int i, len, found, t;

    for (i = 0; i < 5; i++)
    {
        len = strlen(handles[i]);

        if (stack[top] == handles[i][0] && top + 1 >= len)
        {
            found = 1;

            for (t = 0; t < len; t++)
            {
                if (stack[top - t] != handles[i][t])
                {
                    found = 0;
                    break;
                }
            }

            if (found)
            {
                stack[top - t + 1] = 'E';
                top = top - t + 1;
                strcpy(lasthandle, handles[i]);
                stack[top + 1] = '\0';
                return 1;
            }
        }
    }

    return 0;
}

void dispstack()
{
    int j;

    for (j = 0; j <= top; j++)
        printf("%c", stack[j]);
}

void dispinput()
{
    int j;

```

```

    for (j = i; j < l; j++)
        printf("%c", input[j]);
}

int main()
{
    input = (char *)malloc(50 * sizeof(char));

    printf("\nEnter the string:\n");
    scanf("%49s", input);

    strcat(input, "$");
    l = strlen(input);

    strcpy(stack, "$");

    printf("\nSTACK\tINPUT\tACTION");

    while (i <= l)
    {
        shift();

        printf("\n");
        dispstack();
        printf("\t");
        dispinput();
        printf("\tShift");

        while (prec[getindex(stack[top])][getindex(input[i])] == '>')
        {
            if (reduce())
            {
                printf("\n");
                dispstack();
                printf("\t");
                dispinput();
                printf("\tReduced: E->%s", lasthandle);
            }
            else
            {
                break;
            }
        }
    }

    if (strcmp(stack, "$E$") == 0)
        printf("\nAccepted\n");
    else
        printf("\nNot Accepted\n");

    free(input);
}

```

```
    return 0;
}
```

Output:

Enter the string:

i(i+i)*i*

STACK	INPUT	ACTION
\$i	*(i+i)*i\$	Shift
\$E	*(i+i)*i\$	Reduced: E->i
\$E*	(i+i)*i\$	Shift
\$E*(	i+i)*i\$	Shift
\$E*(i	+i)*i\$	Shift
\$E*(E	+i)*i\$	Reduced: E->i
\$E*(E+	i)*i\$	Shift
\$E*(E+i	) *i\$	Shift
\$E*(E+E	) *i\$	Reduced: E->i
\$E*(E+E)	*i\$	Shift
\$E*(E+E)*	i\$	Shift
\$E*(E+E)*i	\$	Shift
\$E*(E+E)*E	\$	Reduced: E->i
\$E*(E+E)*E\$		Shift
\$E*(E+E)*E\$		Shift

Not Accepted

Process finished with exit code 0